

Trabajo Práctico Integrador

Estructuras de Datos, Equals/HashCode y Complejidad Algorítmica

Desarrollo de Software

Fecha: 31/08/2026

Consigna 1: Diagnóstico de Bug y Soluciones

El Problema:

La clase TarjetaAcceso tiene un problema: cuando cambias el nivel de acceso, el objeto se pierde en el HashSet. Esto es porque el nivel forma parte del hashCode(), así que si cambias el nivel, cambia el hash.

Código Original (CON BUG):

```
public class TarjetaAcceso {
    private String cod;
    private int nivel;

    public TarjetaAcceso(String cod, int nivel) {
        this.cod = cod;
        this.nivel = nivel;
    }

    public void setNivel(int nivel) {
        this.nivel = nivel; // PROBLEMA: Cambia el hashCode!
    }

    @Override
    public int hashCode() {
        return Objects.hash(cod, nivel); // Usa los dos
    }
}

// Uso:
Set<TarjetaAcceso> set = new HashSet<>();
TarjetaAcceso t = new TarjetaAcceso("ACC-101", 2);
set.add(t);

t.setNivel(5); // Cambio el hash
set.contains(t); // Devuelve FALSE! ¡Pero está en el set!
```

¿Por qué pasa?

Cuando agregas t con nivel=2, HashSet lo guarda en un 'cajón' basado en ese hash. Cuando cambias el nivel a 5, el hash cambia, pero el objeto sigue en el cajón viejo. Cuando buscas, HashSet calcula el nuevo hash y busca en el cajón nuevo... ¡no lo encuentra!

Solución 1: Hacer la clase IMMUTABLE

```
public final class TarjetaAccesoImmutable {
    private final String cod; // final = no se puede cambiar
    private final int nivel; // final = no se puede cambiar

    public TarjetaAccesoImmutable(String cod, int nivel) {
        this.cod = cod;
        this.nivel = nivel;
    }
}
```

```

    }

    // NO hay setNivel() - punto!
    // El hashCode nunca cambia -> NO hay bug

    @Override
    public int hashCode() {
        return Objects.hash(cod, nivel);
    }
}

```

Solución 2: Usar SOLO 'cod' en el hashCode

```

public class TarjetaAccesoAlternativa {
    private String cod;
    private int nivel;

    @Override
    public boolean equals(Object o) {
        // Solo comparar cod, no nivel
        return Objects.equals(cod, ((TarjetaAccesoAlternativa) o).cod);
    }

    @Override
    public int hashCode() {
        // Solo cod, nunca cambia aunque modifique nivel
        return Objects.hash(cod);
    }

    public void setNivel(int nivel) {
        this.nivel = nivel; // Ahora está permitido
    }
}

```

La **Solución 1** es mejor porque imposibilita el error. La **Solución 2** funciona pero permite cambiar el nivel.

Consigna 2: Elegir las Colecciones Correctas

Para un e-commerce necesitamos guardar datos de formas distintas. Cada forma requiere una colección diferente.

Requerimiento	Interfaz	Implementación	Búsqueda	Inserción
Auditoría (orden exacto)	Queue	LinkedList	$O(n)$	$O(1)$
Ofertas (ordenado por precio)	NavigableMap	TreeMap	$O(\log n)$	$O(\log n)$
Búsqueda de sesiones (500k usuarios)	Map	HashMap	$O(1)$	$O(1)$

Ejemplos:

```
// 1. Auditoría - Queue con LinkedList
Queue<Transaccion> auditoria = new LinkedList<>();
auditoria.add(t1); //  $O(1)$  al final
Transaccion t = auditoria.poll(); //  $O(1)$  del inicio

// 2. Ofertas - TreeMap ordenado
TreeMap<Integer, Producto> ofertas = new TreeMap<>();
ofertas.put(99, laptop);
ofertas.put(49, mouse);
// firstEntry() = producto más barato
// lastEntry() = producto más caro

// 3. Sesiones - HashMap para búsqueda rápida
HashMap<String, Session> sesiones = new HashMap<>();
sesiones.put("token123", session1); //  $O(1)$ 
sesiones.get("token123"); //  $O(1)$  promedio
```

Consigna 3: Optimización $O(n^2) \rightarrow O(n)$

Encontrar el primer carácter que NO aparece dos veces. Ejemplo: 'barco' $\rightarrow$ 'b', 'abarca' $\rightarrow$ 'r'

Versión Lenta: $O(n^2)$

```
static Character lento(String s) {
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        boolean ok = false;

        for (int j = 0; j < s.length(); j++) {
            if (i != j && s.charAt(j) == c) {
                ok = true;
                break;
            }
        }

        if (!ok) {
            return c;
        }
    }
    return null;
}
// Para "abarca" (6 caracteres) = 6*6 = 36 operaciones
```

Versión Rápida: $O(n)$

```
static Character rapido(String s) {
    Map<Character, Integer> m = new HashMap<>();

    // Pasada 1: contar
    for (char c : s.toCharArray()) {
        m.put(c, m.getOrDefault(c, 0) + 1);
    }

    // Pasada 2: encontrar el que aparece 1 vez
    for (char c : s.toCharArray()) {
        if (m.get(c) == 1) {
            return c;
        }
    }
    return null;
}
// Para "abarca" = 6 + 6 = 12 operaciones
```

La versión rápida hace dos pasadas lineales en lugar de dos bucles anidados. Con strings grandes, es MUCHO más rápida.

Consigna 4: Colisiones de Hash

Dos strings distintos pueden tener el mismo hashCode(). Ejemplo: 'FB' y 'Ea' ambos dan 2236.

Paso	Qué pasa	Resultado
1	add('FB')	Bucket[2236]
2	add('Ea')	COLISIÓN
3	equals?	false
4	Resultado	Se guardan AMBOS
5	contains('FB')?	✓ ENCONTRADO

¿Cómo funciona?

1. Calcula hashCode() de ambos strings → 2236 (igual)
2. Va al Bucket[2236]
3. Encuentra 'FB', compara con equals(): 'FB' equals 'FB'? No
4. Continúa buscando en el bucket
5. Encuentra 'Ea', compara: 'Ea' equals 'Ea'? Sí → Encontrado

Conclusión:

Aunque dos objetos tengan el mismo hashCode, Java usa equals() para verificar que sean realmente iguales. Esto se llama 'resolución de colisiones'. En Java 8+, si un bucket tiene más de 8 elementos, cambia a TreeMap para mantener $O(\log n)$.

Resumen Final

Las colecciones de Java están diseñadas para manejar estos casos. Lo importante es elegir la correcta según la operación principal: LinkedList para orden FIFO, TreeMap para elementos ordenados, HashMap para búsqueda rápida.